Viva Preparation

Experiments 6, 7, and 8

Data Science Lab | Questions follow each experiment | Ojas Srivastava, U24AI028

Hadoop on this Mac (needed before Experiment 6)

Install once

- brew install openjdk@17
- Download Hadoop 3.4.1 and extract it to /Users/ojas/hadoop-3.4.1
- Keep it outside the DS Lab folder (the space in that path breaks Hadoop)

Every time you open a terminal

- export JAVA_HOME=/opt/homebrew/opt/openjdk@17/libexec/openjdk.jdk/Contents/Home
- export HADOOP_HOME=/Users/ojas/hadoop-3.4.1
- export PATH="$JAVA_HOME/bin:$HADOOP_HOME/bin:$HADOOP_HOME/sbin:$PATH"
- Check with java -version and hadoop version

Start HDFS

- core-site.xml: fs.defaultFS = hdfs://localhost:9000
- hdfs-site.xml: replication 1, NameNode and DataNode folders
- hdfs namenode -format once per fresh run (this wipes HDFS)
- hdfs --daemon start namenode
- hdfs --daemon start datanode
- Check with jps (NameNode, DataNode)

Stop HDFS

- hdfs --daemon stop datanode
- hdfs --daemon stop namenode

Experiment 6: Word Count using MapReduce

Aim

Count how often each word appears in a text file, using a Hadoop MapReduce program.

What you did

- Started HDFS and uploaded input.txt to /user/lab6/input.
- Wrote WordCount.java: a Mapper, a Reducer, and a Driver.
- Compiled it with javac using the Hadoop classpath and packed it into wordcount.jar.
- Ran: hadoop jar wordcount.jar WordCount <input> <output>
- Read the result with hdfs dfs -cat <output>/part-r-00000

What to say about the program

- Mapper: reads one line, splits it into words, and emits (word, 1).
- Shuffle and Sort: Hadoop groups every value for the same word and sends them to one Reducer.
- Reducer: adds the 1s and writes (word, total).
- The same Reducer is also the Combiner, so counts are added on the map side before data is sent across.
- The Driver sets the Mapper, Reducer, input path, and output path, then submits the job.

Result for input.txt

- 14 words, 10 unique. hadoop, big, data, and mapreduce appear twice. The rest appear once.
- Output file: part-r-00000. _SUCCESS means the job finished.
- The output folder must not already exist. Delete it with hdfs dfs -rm -r before running again.

Exercises

Q: 1. Modify the input and observe the output.

A: Two lines were added. Words went from 14 to 28, unique words from 10 to 18. hadoop went from 2 to 4.

Q: 2. Ignore common words such as is, the, and a.

A: Run with -D wordcount.skip.stopwords=true. The Mapper skips those words. Map output records dropped from 28 to 22.

Q: 3. Run the program on another text file.

A: another.txt gave 39 words and 29 unique. The same JAR works on any text file.

Q: 4. Compare the output before and after changing the input.

A: Some words stayed the same. hadoop, is, big, and data increased. New words appeared, such as hdfs, blocks, and scalable.

Q: 5. Explain the Mapper and the Reducer.

A: The Mapper emits (word, 1) for each word and does not count across lines. The Reducer adds those 1s to get the final count.

Questions from this experiment

Q: What is MapReduce?

A: A way to process data in parallel: Map produces (key, value) pairs, Reduce combines the values for each key.

Q: What does the Mapper emit here?

A: (word, 1) for every word.

Q: What does the Reducer do?

A: It sums the counts for one word and writes (word, total).

Q: What is the Combiner?

A: A local reduce on the map side. It cuts down how much data is sent to the Reducer.

Q: What is Shuffle and Sort?

A: The step that groups values by key and sends each group to one Reducer.

Q: Why Text and IntWritable?

A: Hadoop needs these types to send data between machines.

Q: What is part-r-00000?

A: The file written by reducer number 0.

Q: Where is the output stored?

A: In HDFS, in the output directory you pass to the job.

Experiment 7: Data processing with Apache Pig

Aim

Analyze the Titanic dataset with Apache Pig. Pig Latin is turned into MapReduce jobs, so you do not write Java.

Dataset

Titanic-Dataset.csv, 891 passengers. Columns used: Survived, Pclass, Sex, Age, Fare, Embarked.

What you did

- Started HDFS and uploaded the CSV to /user/lab7/titanic.
- Ran the script in MapReduce mode: pig -x mapreduce script.pig
- LOAD the file, FILTER survivors, GROUP by Pclass, COUNT passengers, ORDER the result, STORE it in HDFS.

Operators in the script

- LOAD: read the CSV into a relation. CSVExcelStorage is used because names contain commas inside quotes.
- DUMP: print a relation. STORE: save it to an HDFS directory.
- FILTER: keep rows that match a condition.
- GROUP: collect rows that share a key, for example the same Pclass.
- FOREACH: produce new fields, such as the count and the average fare.
- ORDER: sort the result.

Result

Pclass	Passengers	Survived	Avg fare	Survival %
1	216	136	84.15	62.96
2	184	87	20.66	47.28
3	491	119	13.68	24.24

Survivors: 342. Female passengers: 314.

Exercises

Q: 1. Filter only female passengers.

A: FILTER titanic BY Sex == 'female'. Result: 314 rows.

Q: 2. Count passengers in each class.

A: GROUP by Pclass, then COUNT. Class 1: 216, class 2: 184, class 3: 491.

Q: 3. Sort passengers by Fare, descending.

A: ORDER BY Fare DESC. Highest fare is 512.33. Lowest fares are 0.0.

Q: 4. Store the output in a new HDFS directory.

A: STORE ... INTO '/user/lab7/exercises/ex4_female_summary'. The directory has part-r-00000 and _SUCCESS. Female survival: 1st class 96.81%, 2nd 92.11%, 3rd 50%.

Q: 5. Difference between FILTER and FOREACH.

A: FILTER keeps or drops rows. FOREACH changes the columns of each row. Age > 70 kept 5 rows. FOREACH on those rows kept 5 rows but only 4 columns.

Questions from this experiment

Q: What is Apache Pig?

A: A tool on Hadoop. You write Pig Latin, and Pig runs it as MapReduce jobs.

Q: Why use Pig instead of Java MapReduce?

A: The same work takes a few lines. FILTER, GROUP, and ORDER are built in.

Q: What is a relation?

A: Pig's table: a collection of rows (tuples).

Q: Local mode versus MapReduce mode?

A: Local mode reads the local disk. MapReduce mode reads HDFS and runs Hadoop jobs. This experiment uses MapReduce mode.

Q: DUMP versus STORE?

A: DUMP prints on the screen. STORE writes a directory in HDFS.

Q: What is lazy evaluation?

A: Nothing runs until DUMP or STORE. The earlier lines only build a plan.

Q: Why CSVExcelStorage?

A: Passenger names contain commas inside quotes. A plain comma split would break those names.

Experiment 8: Query processing with Apache Hive

Aim

Query the Titanic dataset with HiveQL, which is similar to SQL. Hive turns the queries into MapReduce jobs.

Dataset

Same Titanic file, 891 passengers, loaded from HDFS.

What you did

- Started HDFS and uploaded Titanic-Dataset.csv to /user/lab8/titanic.
- Created the database titanic_db.
- Created a staging table titanic_raw and loaded the CSV with LOAD DATA INPATH.
- Created a typed table titanic from that data and ran SELECT queries in Beeline.

What to say about the tables

- titanic_raw reads the CSV. OpenCSVSerde keeps commas that sit inside quoted names.
- titanic is the table used for analysis. Age and Fare are numbers, so AVG and MAX work.
- LOAD DATA only moves the file into the table. Hive applies the column types when the query runs.

Result

- Same class counts as Pig: 216, 184, and 491 passengers.
- Females: 314 passengers, 74.20% survived. Males: 577 passengers, 18.89% survived.
- Survivors: 342 (38.38%).
- Oldest: Barkworth, age 80. Youngest: 0.42 years. 177 passengers have no age.
- Highest fare 512.33. Lowest fare 0.0.

Exercises

Q: 1. Display only female passengers.

A: SELECT ... FROM titanic WHERE sex = 'female'. Result: 314 rows.

Q: 2. Average fare for each passenger class.

A: SELECT pclass, AVG(fare) FROM titanic GROUP BY pclass. Class 1: 84.15, class 2: 20.66, class 3: 13.68.

Q: 3. Count passengers who survived.

A: SELECT COUNT(*) FROM titanic WHERE survived = 1. Result: 342.

Q: 4. Sort passengers by age, descending.

A: SELECT ... FROM titanic WHERE age IS NOT NULL ORDER BY age DESC. 714 rows. Oldest is 80.

Q: 5. Maximum and minimum fare.

A: SELECT MAX(fare), MIN(fare) FROM titanic. Maximum 512.33, minimum 0.0.

Questions from this experiment

Q: What is Apache Hive?

A: A data warehouse on Hadoop. You query HDFS data with SQL-like HiveQL.

Q: What is the metastore?

A: The place Hive stores table names, column types, and the HDFS location of the data. This lab uses Derby.

Q: What does LOAD DATA INPATH do?

A: It moves the file from HDFS into the table. It does not parse the file.

Q: Why OpenCSVSerde?

A: Names contain commas inside quotes. The default comma split would break them.

Q: Managed versus external table?

A: Dropping a managed table deletes the data. Dropping an external table removes only the table definition.

Viva Preparation - Experiments 6, 7, and 8

Q: How does a query run?

A: Hive turns HiveQL into a MapReduce job and runs it on Hadoop.

Q: Pig versus Hive, from these two experiments?

A: Pig scripts list the steps (LOAD, FILTER, GROUP, STORE). Hive queries say what you want (SELECT, WHERE, GROUP BY). Both run on Hadoop.